IMDb Movie Review Sentiment Analysis

Benchmarking Classical vs. Neural Approaches

Sanidhya Kaushik & Agam Ghotiya

Indian Institute of Science, Bangalore

Outline

1. Introduction & Motivation
2. Dataset & Preprocessing
3. Qualitative Analysis
4. Methodology
5. Results
6. Discussion
7. Conclusion

Introduction & Motivation

Problem Statement

Classify IMDb reviews as **positive** or **negative** using both classical ML and deep learning.

Key Questions

- Do neural models outperform TF-IDF?
- Impact of **Global Max Pooling**?
- Discriminative textual features?

Dataset: IMDb

- **50,000** samples (Balanced)
- Long-form (avg. 230 words)
- Binary sentiment labels

Why It's Challenging

- Noisy/informal language
- Variable long sequences
- Sarcasm & mixed sentiment

Dataset & Preprocessing Pipeline

Text Preprocessing Pipeline

1. **Normalization**: Conversion to lowercase.
2. **Cleaning**: Removal of HTML tags and punctuation via RegEx.
3. **Tokenization**: Splitting text into word units.
4. **Classical Modeling**: TF-IDF vectorization (max 5K–20K features), inclusion of bigrams.
5. **Deep Learning**: Vocabulary of **10,000 tokens**, sequences padded/truncated to **length 250**.

Dataset & Preprocessing: Insights

Exploratory Observations

- Both positive & negative reviews show **similar length distributions**.
- Average word length is **identical** across sentiments ($\approx$ 5.7 chars).
- Word count peaks near 200 words per review.

Key Conclusion

Structural features (length, word size) are not discriminative. The sentiment signal lies strictly in **lexical content** (word choice).

Exploratory Data Analysis: Review Length & Word Length

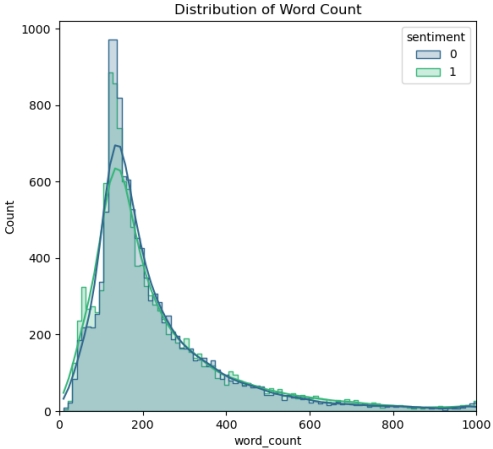


Fig 1: Word count distribution by sentiment.

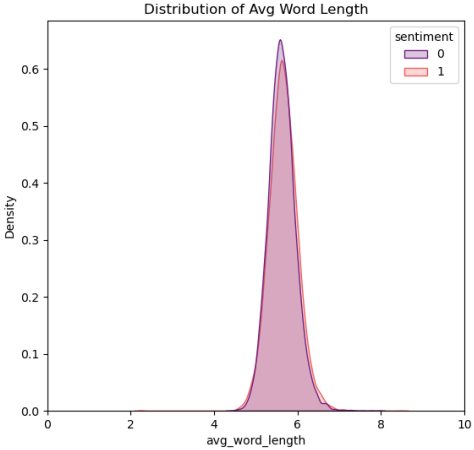


Fig 2: Average word length distribution.

Qualitative Analysis: Word Clouds

Negative Reviews Importance

Negative: *bad, worst, boring, waste, awful.*

Positive Reviews Importance

Positive: *great, love, best, excellent, story.*

Observation

Discriminative tokens are highly sentiment-specific, confirming that TF-IDF can capture meaningful signal.

Qualitative Analysis: Word Polarity & LSA

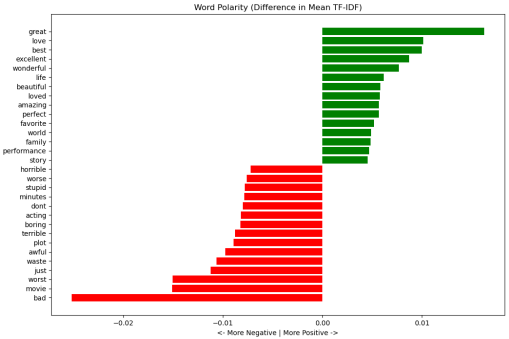


Fig 5: Word polarity via difference in mean TF-IDF scores.

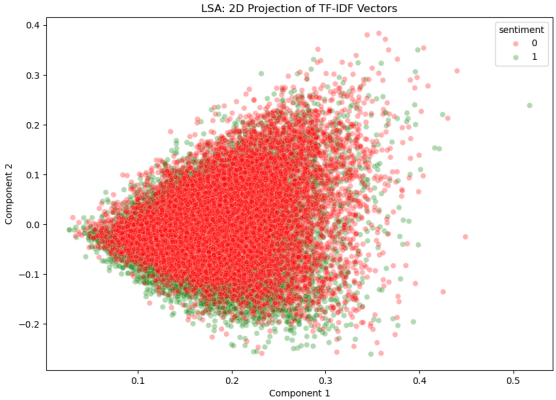


Fig 6: LSA 2D projection of TF-IDF vectors (High overlap).

Methodology: Models Overview

Classical Models (TF-IDF)

- **Naive Bayes (NB)**
- **Logistic Regression (LR)**
- **Linear SVM**

Hyperparameter Grid Search:

- Regularization strength C / α
- TF-IDF max features (5K–20K)
- N-gram range (unigrams, bigrams)

Neural Models (Sequence)

- **Vanilla RNN** + Max Pooling
- **LSTM** + Max Pooling

Architecture:

- Embedding (Vocab 10K, Seq 250)
- **Global Max Pooling**

Hyperparameter Tuning: Results and Search Space

Logistic Regression

- **Best Score:** 0.8844
- **C:** 1.0
- **Penalty:** L2
- **Features:** 10,000
- **N-grams:** (1, 2)

Naive Bayes

- **Best Score:** 0.8682
- **Alpha:** 1.0
- **Features:** 20,000
- **N-grams:** (1, 2)

Linear SVM

- **Best Score:** 0.8864
- **C:** 0.1
- **Features:** 10,000
- **N-grams:** (1, 1)

Grid Search Summary

A total of **230 fits** were performed via 5 fold cross validation. Tuning Linear SVM provided the highest relative gain (+2.1%) compared to baseline settings.

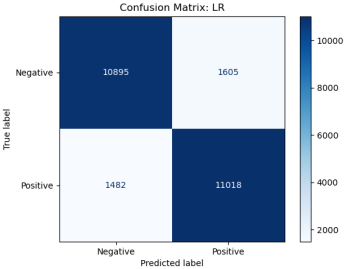
Quantitative Results: All Models

Model	Accuracy	Precision	Recall	F1-Score
LR (Tuned)	**88.00%**	**0.8664**	**0.8983**	**0.8821**
SVM (Tuned)	87.90%	0.8682	0.8937	0.8808
LR Baseline	87.65%	0.8729	0.8814	0.8771
LSTM	85.23%	0.8556	0.8331	0.8442
SVM Baseline	85.79%	0.8594	0.8558	0.8576
NB (Tuned)	84.90%	0.8526	0.8440	0.8483
RNN	81.23%	0.8523	0.7556	0.8010

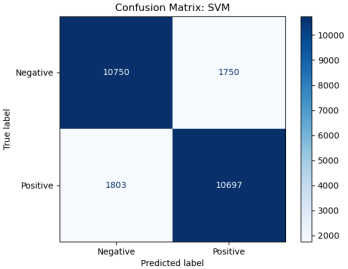
Summary Findings

Tuned Logistic Regression achieved the best performance. Deep learning models without pre training struggled to outperform classical linear models.

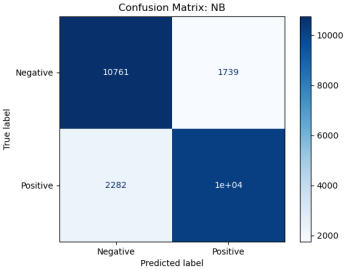
Confusion Matrices: Classical Models (Baseline)



Logistic Regression

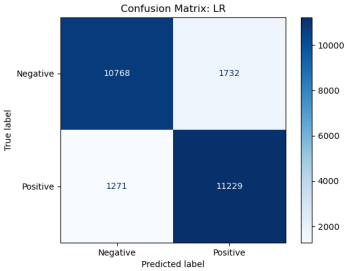


Linear SVM

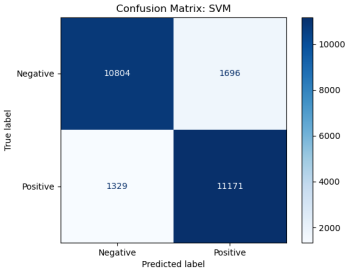


Naive Bayes

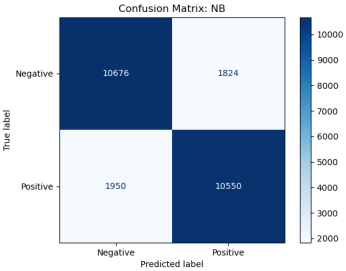
Confusion Matrices: Classical Models (Tuned)



LR (Tuned)



SVM (Tuned)

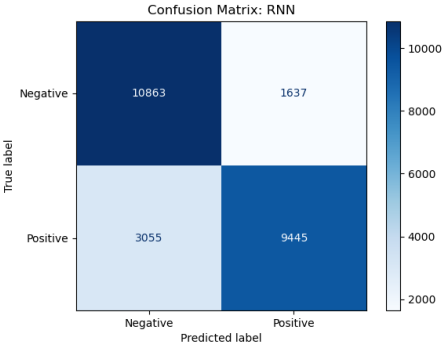


NB (Tuned)

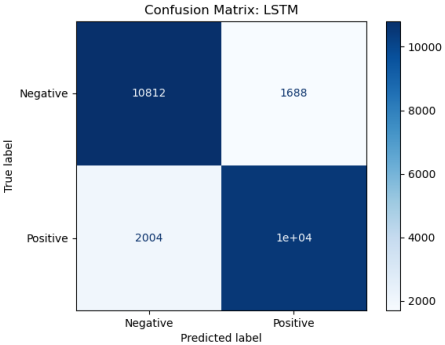
Gains from Tuning

SVM benefited most (+2.1% accuracy) by optimizing the regularization parameter C and max features.

Confusion Matrices: Neural Models

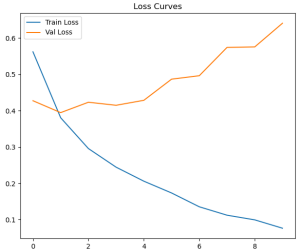


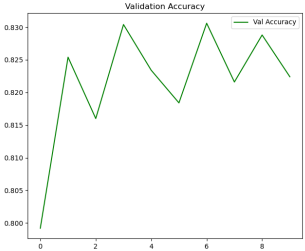
Vanilla RNN (81.23%)



LSTM (85.23%)

Neural Models: RNN Training Curves

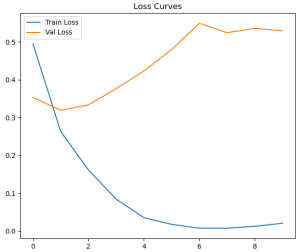


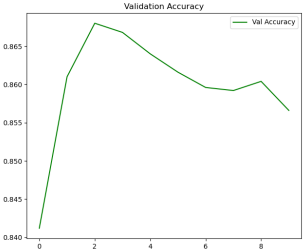


Vanilla RNN Analysis

- Training loss decreases sharply but **validation loss diverges**.
- Clear sign of overfitting after Epoch 4.
- Accuracy fluctuates around 81–83%.

Neural Models: LSTM Training Curves





LSTM Analysis

- Much more stable convergence.
- Validation loss plateaus early ($\approx$ Epoch 2).
- Accuracy peaks at $\approx$ **86.9%**.

Why Do Classical Models Outperform Neural Models?

Reasons for Neural Underperformance

1. **No Pretraining**: Models were trained from scratch without BERT or GloVe.
2. **Information Bottleneck**: Sequential flow is less useful than lexical importance for this specific dataset.

Why Linear SVM/LR Worked

- **Global Context**: TF-IDF counts all tokens, whereas RNNs can "forget" start of review tokens.
- **Optimization**: Linear models find global minima in convex spaces; neural models can get stuck.

Global Max Pooling: Role in Recurrent Architectures

The Problem: Information Loss

Standard RNNs pass only the **final hidden state**. Padding tokens dilute this state, leading to $\approx 50\%$ accuracy on long sequences.

The Solution: Max Pooling

$$h^* = \max_{t=1}^{T} h_t$$

Captures the most *salient* activation across all timesteps, regardless of position.

Performance Impact

- **RNN**: Essential (0% learning without it).
- **LSTM**: Improved recall by reducing padding noise.

Summary

- **Best Model**: Tuned Logistic Regression (88.0% Accuracy).
- Classical TF-IDF models consistently outperform RNNs/LSTMs trained from scratch here.
- Word choice is more informative than sequence order for IMDb.

Thank You

Questions & Discussion

Sanidhya Kaushik
sanidhyak@iisc.ac.in

Agam Ghotiya
agamghotiya@iisc.ac.in